
ANALYSE, MODÉLISATION, DOCUMENTATION ET CONCEPTION : COMBAT DE ROBOTS

Développement d'applications (Expert)

2026-09-11

CÉGEP DE CHICOUTIMI

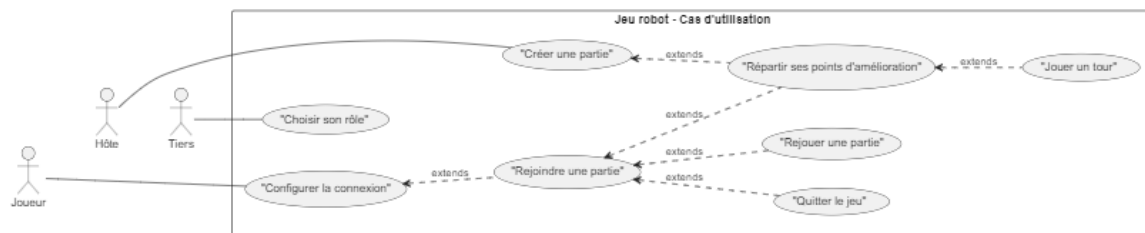
Groupe 3

(Henri Pitre, Étienne Tremblay, Thomas Derdeyn et Mathieu Tenret)

Table des matières :

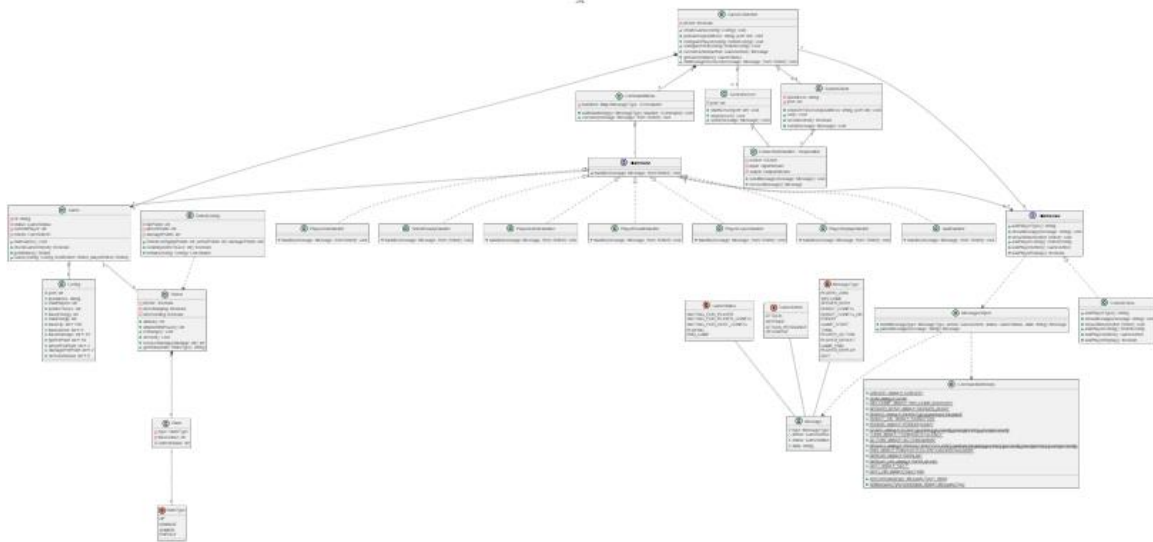
Table des matières :	1
1. Diagramme des uses cases :	2
2. Diagramme de classes métier :	3
Diagramme :	4
Légende des relations :	4
Explication du diagramme :	4
3. Diagramme séquence	7
4. Scénarios	9
4. Échéancier :	9
5. Planification du plan de tests :	9
6. Remarque.....	9

1. Diagramme des uses cases :



2. Diagramme de classes métier :

Diagramme :



Légende des relations :

Association bidirectionnelle : Classe associée « x » — « y » Classe associée

Association unidirectionnelle : Classe référencée (cible) <— « y » Classe qui référence (source)

Héritage : Classe mère <|— Classe enfant

Agrégation : Agrégat « x » ◇— « y » Partie (composant)

Composition : Composite « x » ◆— « y » Partie (composant)

Dépendance : Dépendance <-- Classe dépendante

Implémentation d'interface : Interface implémentée <|-- Classe qui l'implémente

Explication du diagramme :

GameStatus : Cette énumération représente l'état actuel d'une partie. Chaque objet Game utilise ce type pour représenter son état, tandis que chaque Message peut transporter l'état du jeu.

GameAction : Cette énumération représente les actions qu'un joueur peut choisir d'effectuer pendant son tour de jeu [ATTACK, DEFENSE, ATTACK_PUISSANTE, RECHERCHE].

MessageType : Cette énumération définit toutes les catégories de messages.

Config : Cette classe rassemble tous les paramètres nécessaires à la configuration du jeu [port, ipAddress, maxPlayers, pointsToGive, baseEnergy et maxEnergy].

Elle est reliée à l'agrégat Game (Game « 1 » ◇-- « 1 » **Config**); ce qui signifie que chaque objet Game possède une Config, mais que Config n'est pas considérée comme une partie inséparable de Game.

RobotConfig : Cette classe rassemble toutes les caractéristiques de la configuration choisie par le joueur avant le début du combat [hpPoints, armorPoints, damagePoints] et permet de vérifier qu'elle soit correcte.

Cette classe dépend de la classe Robot (Robot « 1 » — « 1 » **RobotConfig**); car elle est utilisée pour valider et construire les statistiques des robots .

Robot : Cette classe représente le robot contrôlé par le joueur.

Cette classe est une composante de la classe composite Game (Game « 1 » ◆— « 2 » **Robot**).

D'autre part, cette classe est également le composite de la classe Stats (**Robot** « 1 » ◆— « 4 » Stats); ce qui signifie que chaque Robot est constitué de quatre Stats [HP, ATTACK, DEFENSE et ENERGY].

Stats : Cette classe représente une caractéristique du robot [HP, ATTACK, DEFENSE ou ENERGY].

Cette classe est une composante de la classe composite Robot (Robot « 1 » ◆— « 4 » **Stats**).

D'autre part, cette classe est associée à la classe StatsType (StatsType « 1 » — « * » **Stats**); ce qui signifie que chaque Stats possède exactement un StatsType.

StatsType : Cette énumération détermine ce que chaque objet Stats représente [HP, ATTACK, DEFENSE ou ENERGY].

Comme mentionné au-dessus, cette classe est associée à la classe Stats (**StatsType** « 1 » — « * » Stats).

Game : Cette classe représente une partie.

Comme mentionné plus tôt, cette classe est associée à la classe Robot (**Game** « 1 » — « 2 » Robot).

De plus, cette classe est une composante de la classe composite GameController (GameController « 1 » ◆— « 1 » **Game**); ce qui signifie que chaque GameController possède exactement une Game, et que Game est une partie intégrante de GameController.

GameController : Cette classe permet d'orchestrer les parties. Elle fait le lien entre la partie, l'interface utilisateur et les différents traitements des messages reçus.

Cette classe est le composite de la classe Game (**GameController** « 1 » ◆— « 1 » Game); ce qui signifie que chaque GameController possède exactement une Game, et que Game est une partie intégrante de GameController.

D'autre part, cette classe est l'agrégat de la classe SocketServer (**GameController** « 1 » ◇— « 0..1 » SocketServer); ce qui signifie que chaque GameController peut posséder 0 ou 1 SocketServer. Le SocketServer est utilisé lorsque le joueur joue le rôle de serveur.

Cette classe est également l'agrégat de la classe SocketClient (**GameController** « 1 » ◇— « 0..1 » SocketClient); ce qui signifie que chaque GameController peut posséder 0 ou 1 SocketClient. Le SocketClient est utilisé lorsque le joueur joue le rôle de client.

Ensuite, cette classe est associée à l'interface IGameView (**GameController** « 1 » — « 1 » IGameView) ; ce qui signifie que chaque GameController utilise exactement une interface de jeu IGameView.

Pour finir, cette classe est le composite de la classe CommandMenu (**GameController** « 1 » ◆— « 1 » CommandMenu); ce qui signifie que chaque GameController possède exactement un CommandMenu, et que CommandMenu est une partie intégrante de GameController.

Message : Cette classe permet d'échanger les informations entre client et serveur à travers la connexion TCP. Cette classe est utilisée par la classe MessageHelper, qui permet de transformer un Message en données transmissibles sur le réseau, ou inversement de transformer les données reçues en objet Message.

CommandDictionary : Cette classe contient les commandes textuelles utilisées dans les échanges entre client et serveur.

MessageHelper : Cette classe sert d'intermédiaire entre les objets Message du programme et leurs données transmissibles sur le réseau. Elle permet donc de construire un Message et d'interpréter les données reçues.

Cette classe dépend de la classe CommandDictionary (**MessageHelper** --> CommandDictionary); ce qui signifie qu'elle utilise les commandes définies dans CommandDictionary.

Elle dépend également de la classe Message (**MessageHelper** --> Message) ; ce qui signifie qu'elle crée, lit ou manipule les objets Message.

SocketClient: Cette classe représente le Socket qui gère la partie réseau du client.

SocketServer: Cette classe représente le Socket qui gère la partie réseau du serveur.

ConnectionHandler : Cette classe représente une connexion réseau active entre deux programmes. Elle utilise un Socket ainsi que des flux d'entrée et de sortie afin d'envoyer et de recevoir des messages.

IGameView : Interface qui définit le contrat que doit respecter n'importe quelle interface du jeu.

ConsoleView : Cette classe représente l'interface utilisateur. Elle permet notamment d'afficher des messages et de récupérer les choix du joueur.

ICommand : Interface qui définit le contrat que doivent respecter toutes les classes chargées de traiter un message reçu. Chaque ICommand accède à la classe Game pour appliquer les effets du message, et peut aussi utiliser une vue via l'interface IGameView pour afficher un résultat

Les différents Handler : Ces classes permettent chacune de traiter un type précis de message reçu.

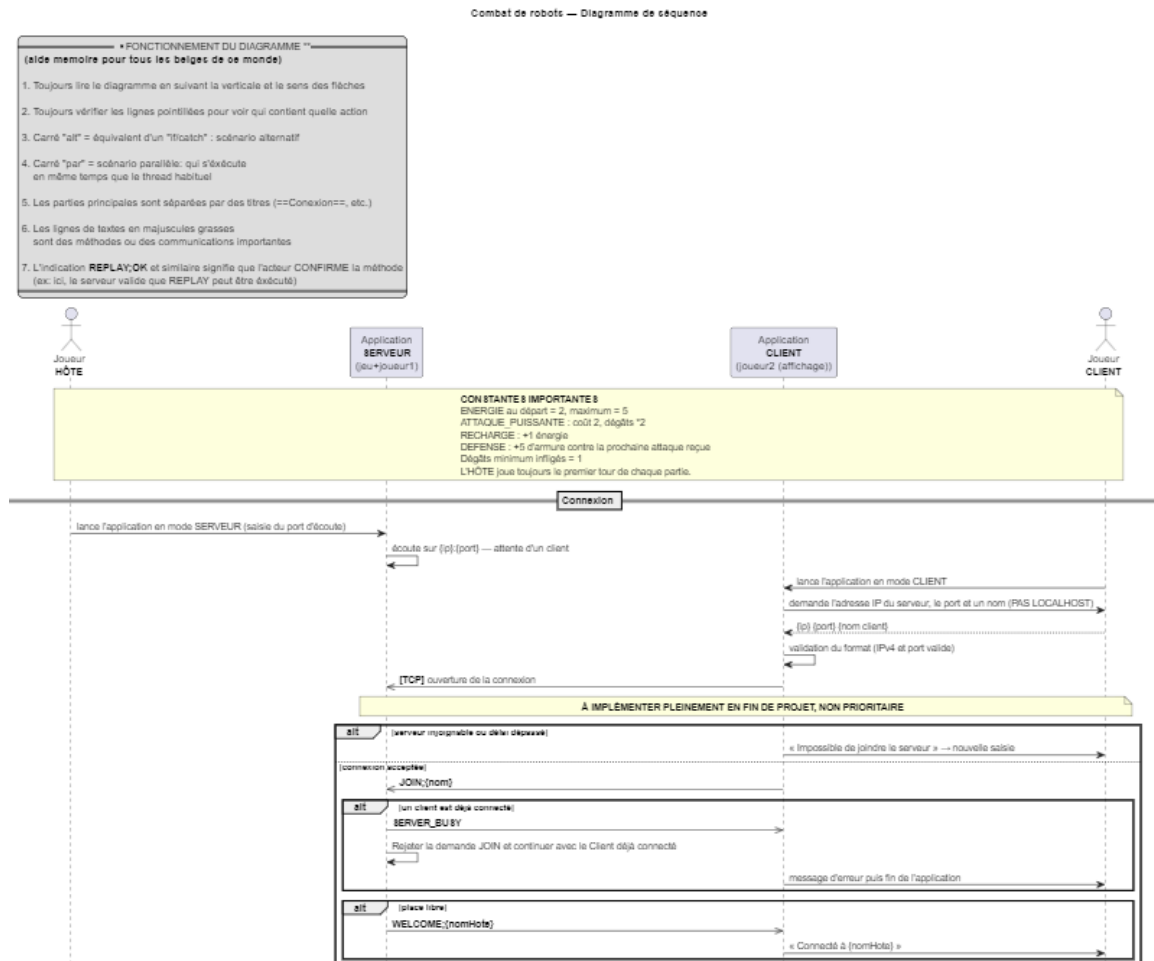
Par exemple, PlayerJoinHandler traite l'arrivée d'un joueur, RobotReadyHandler traite la confirmation de la configuration du robot d'un joueur, PlayerActionHandler traite une action du combat, PlayerResultHandler traite le résultat de ces actions, PlayerLeaveHandler traite le départ d'un joueur, PlayerReplayHandler traite une demande de nouvelle partie et QuitHandler traite la fermeture de la connexion.

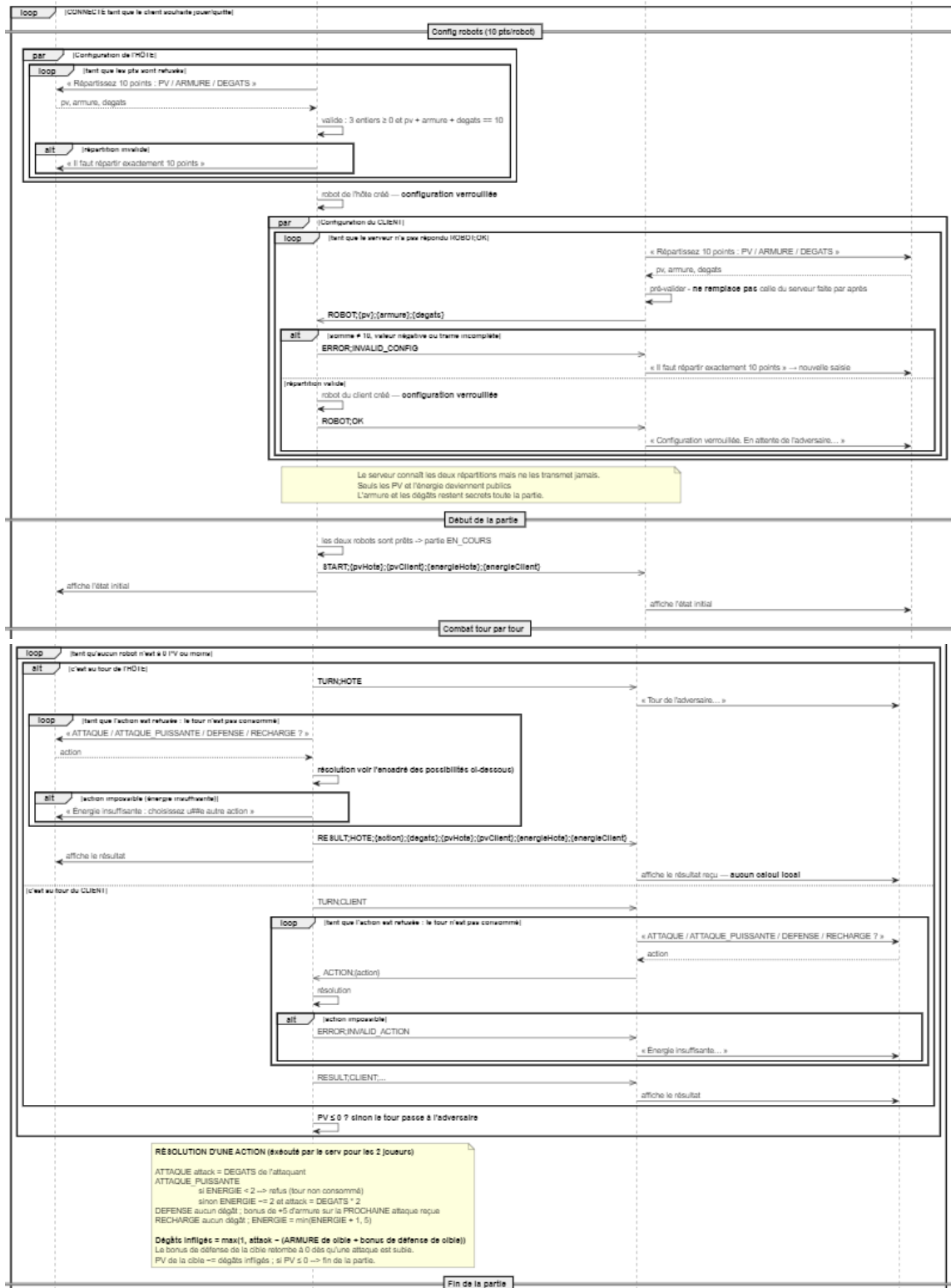
Toutes ces classes implémentent l'interface ICommand (ICommand <|-- **Handler**); ce qui signifie qu'elles doivent toutes posséder une méthode « handle ».

CommandMenu : Cette classe permet de choisir quel Handler doit être utilisé lorsqu'un Message est reçu. Elle contient une collection qui associe chaque MessageType à un ICommand.

Par exemple, un Message de type PLAYER_ACTION sera associé à PlayerActionHandler, tandis qu'un Message de type PLAYER_JOIN sera associé à PlayerJoinHandler.

3. Diagramme séquence





4. Scénarios

Nom : Choisir son rôle

Identifiant : UC-01

Acteurs : Tiers

Description : Cet use case permet à chaque utilisateur de choisir le rôle qu'il souhaite occuper dans le jeu (joueur serveur, c'est-à-dire hôte, ou joueur client).

Préconditions :

1. L'utilisateur a lancé l'application.
2. Aucun rôle n'a encore été sélectionné.
3. L'application écoute sur son adresse IP et le port déterminé via la config.

Scenario nominal :

1. Le système affiche un message pour savoir si le joueur veut créer ou rejoindre une partie (Ce qui correspond aux deux rôles disponibles : hôte ou joueur).
2. L'utilisateur choisit de créer une partie (rôle de joueur serveur).
3. Le système enregistre l'utilisateur comme joueur serveur.
4. Le système crée la partie.

Scénario alternatif :

CAS A : L'utilisateur choisit le rôle de client.

- 2a. L'utilisateur choisit de rejoindre une partie.
- 3a. Le système enregistre l'utilisateur comme joueur client.
- 4a. Le système permet au joueur de configurer sa connexion.

CAS B : Choix invalide.

- 2b. L'utilisateur saisit un choix invalide.
- 3b. Le système affiche un message d'erreur.
- 4b. Retour à l'étape 1 du scénario nominal.

Postconditions :

Succès : Le rôle de l'utilisateur est défini comme joueur serveur ou joueur client.

Échec : Aucun rôle valide n'a été sélectionné. L'utilisateur ne peut ni créer ni rejoindre de partie.

Nom : Créer une partie

Identifiant : UC-02

Acteurs : Hôte

Description : Cet use case permet à l'hôte de créer une partie et d'attendre la connexion d'un autre joueur (client).

Préconditions :

1. Le joueur a choisi le rôle d'hôte.
2. L'application est en mesure de démarrer une connexion serveur.

Scénario nominal :

1. L'hôte demande la création d'une partie.
2. Le système récupère les paramètres nécessaires à la création d'une partie.
3. Le système démarre le socket serveur.
4. Le socket serveur écoute sur le port configuré.
5. Le système crée la partie.
6. Le socket serveur attend qu'un joueur s'y connecte.
7. La connexion est établie, les joueurs peuvent configurer leurs robots.

Scénario alternatif :

CAS A : L'application n'est pas en mesure de démarrer une connexion serveur.

3a. Une erreur survient lors de l'ouverture du socket serveur.

4a. Le système affiche un message d'erreur pour indiquer que la partie ne peut pas être créée.

CAS B : Aucun client ne se connecte.

4a. Aucun joueur client ne se connecte dans le délai imparti.

5a. Le système affiche un message d'erreur pour indiquer que la connexion est interrompue.

Postconditions :

Succès : La partie est créée.

Échec : La partie ne peut pas être créée. L'hôte peut réessayer.

Cas d'extension : « Répartir ses points d'amélioration » (UC-05).

Nom : Configurer la connexion

Identifiant : UC-03

Acteurs : Joueur

Description : Cet use case permet au joueur client de renseigner les informations nécessaires à la connexion au serveur.

Préconditions :

1. Le joueur a choisi le rôle de joueur.
2. La partie doit avoir été créée par l'hôte pour qu'une connexion puisse être établie avec elle.

Scénario nominal :

1. Le système demande l'adresse IP et le port du serveur.
2. Le joueur saisit l'adresse IP et le port du serveur.

3. Le système vérifie le format des informations saisies par le joueur.
4. Si elles sont correctement saisies, le système les enregistre.
5. Le joueur peut rejoindre la partie.

Scénario alternatif :

CAS A : L'adresse IP ou le port est invalide.

2a. Le joueur saisit un format d'adresse IP ou de port invalide.

3a. Le système affiche un message d'erreur et demande de saisir à nouveau les informations erronées.

4a. Retour à l'étape 1 du scénario nominal.

Postconditions :

Succès : Le joueur peut rejoindre la partie.

Échec : Le joueur ne peut pas rejoindre la partie.

Cas d'extension : « Rejoindre une partie » (UC-04).

Nom : Rejoindre une partie

Identifiant : UC-04

Acteurs : Joueur

Description : Cet use case permet de rejoindre une partie créée par un joueur hôte.

Préconditions :

1. Le joueur a configuré sa connexion (adresse IP et port du serveur).
2. Une partie disponible a été créée par l'hôte et attend une connexion.

Scénario nominal :

1. Le joueur demande à rejoindre la partie.
2. Le système utilise les paramètres de connexion configurés pour tenter de contacter le serveur.
3. L'hôte reçoit la demande de connexion du joueur.
4. L'hôte accepte la connexion du joueur.
5. Une connexion TCP est établie entre le client et le serveur.
6. Le système ajoute le joueur à la partie.
7. Le système récupère les informations nécessaires à la partie.
8. Le système retourne un message au joueur pour lui confirmer qu'il a bien rejoint la partie.

Scénario alternatif :

CAS A : L'hôte est inaccessible.

3a. Le système ne parvient pas à contacter le serveur.

4a. Le système informe le joueur que la connexion a échoué.

5a. Le joueur doit modifier sa configuration ou effectuer une nouvelle tentative.

CAS B : La connexion est refusée.

4a. L'hôte refuse la connexion.

5a. Le système affiche un message pour informer le joueur qu'il ne peut pas rejoindre la partie.

Postconditions :

Succès : Le joueur a bien rejoint la partie.

Échec : Aucune extension n'a pu être établie.

Cas d'extension : « Répartir ses points d'amélioration » (UC-05), « Rejouer une partie » (UC-7), « Quitter le jeu » (UC-8).

Notes : Une fois connecté, le joueur peut rester connecté pour plusieurs parties.

Nom : Répartir ses points d'amélioration

Identifiant : UC-05

Acteurs : Joueur et hôte.

Description : Cet use case permet à chaque joueur de répartir ses points d'amélioration entre les différentes caractéristiques de son robot (HP, armure et dégâts).

Préconditions :

1. Les deux joueurs sont connectés à la partie.
2. Le combat n'a pas encore commencé (impossible de réaliser cette fonctionnalité pendant le combat).

Scenario nominal :

1. Le système indique au joueur qu'il peut répartir 10 points d'amélioration.
2. Le joueur sélectionne le nombre de points qu'il attribue aux HP.
3. Le joueur sélectionne le nombre de points qu'il attribue à son armure.
4. Le joueur sélectionne le nombre de points qu'il attribue aux dégâts.
5. Le joueur confirme sa configuration.
6. Le système vérifie que le nombre total de points attribués est bien de 10.
7. Si c'est bien le cas, le système ajoute 10 HP au joueur par point investi en HP, 2 d'armure par point investi en armure et 2 de dégâts par point investi en dégâts.
8. Le système indique que le joueur est prêt à débiter le combat.

Scénario alternatif :

CAS A : Le nombre de points attribués est incorrect.

7a. Le nombre de points attribués n'est pas de 10.

8a. Le système retourne un message d'erreur pour indiquer que le nombre de points attribués est incorrect.

9a. Retour à l'étape 1 du scénario nominal.

Postconditions :

Succès : Le joueur est prêt pour le combat.

Échec : La configuration est invalide, le joueur ne peut pas débiter le combat.

Cas d'extension : « Joueur un tour » (UC-06).

Notes : La configuration d'un joueur doit rester secrète pour son adversaire avant le début du combat.

Nom : Jouer un tour

Identifiant : UC-06

Acteurs : Joueur et hôte

Description : Cet use case permet au joueur actif de choisir une action.

Préconditions :

1. Les deux joueurs sont prêts (configurations valides).

Scénario nominal :

1. Le système indique au joueur actif que c'est son tour.
2. Le système indique au joueur actif les actions disponibles.
3. Le joueur actif sélectionne une action.
4. (L'action est transmise au serveur si elle provient du client.)
5. Le serveur vérifie que l'action est valide.
6. Le serveur calcule le résultat de l'action et met à jour l'état des robots.
7. Le serveur indique aux deux joueurs le résultat de l'action.
8. Si aucun robot n'est passé à 0 HP ou moins, le tour passe au joueur suivant.

Scénario alternatif :

CAS A : L'action est invalide.

5a. Le système détecte une action invalide.

6a. Le système retourne un message d'erreur.

7a. Retour à l'étape 2 du scénario nominal.

CAS B : La partie est terminée.

8a. L'un des deux robots passe à 0 HP ou moins.

9a. Le système indique aux deux joueurs le gagnant de la partie.

Postconditions :

Succès : L'état des robots a été mis à jour et le tour est passé au joueur suivant, ou la partie est terminée.

Échec : L'état des robots n'a pas pu être modifié, le tour est toujours en attente.

Nom : Rejouer une partie

Identifiant : UC-07

Acteurs : Joueur et hôte.

Description : Cet use case permet aux joueurs de recommencer une partie.

Préconditions :

1. La partie est terminée.
2. La connexion est toujours établie.

Scenario nominal :

1. Le joueur choisit de rejouer.
2. Le système transmet la demande du client au serveur (message « Replay »).
3. Le serveur reçoit le message.
4. Le serveur accepte la demande.
5. Le système crée la nouvelle partie.
6. L'état des robots est réinitialisé.
7. Les joueurs peuvent configurer leur robot.

Scénario alternatif :

CAS A : La connexion est perdue.

2a. La connexion entre le client et le serveur n'est plus établie.

3a. Le système indique qu'une nouvelle partie ne peut pas être lancée.

Postconditions :

Succès : La partie est lancée, les joueurs peuvent configurer leur robot.

Échec : Une nouvelle partie ne peut pas être lancée.

Nom : Quitter une partie

Identifiant : UC-08

Acteurs : Joueur et hôte.

Description : Cet use case permet à un joueur quitter la partie.

Préconditions :

1. Le joueur est connecté à une partie.

Scenario nominal :

1. Le joueur choisit de quitter le jeu.
2. Le système transmet la demande du client au serveur (message « Quit »).
3. Le serveur reçoit le message.

4. La connexion TCP est fermée et les ressources utilisées par celle-ci sont libérées.
5. L'application du joueur peut être fermée.

Postconditions :

Succès : Le joueur n'est plus connecté à la partie.

4. Échéancier :

Date	À faire
2 jours (19 sept)	Connexion (client et serveur fonctionnels, connexion fonctionnelle)
1 jour (20 sept)	Instancier classes, interfaces, séquences
3 jours (23 sept)	Mécaniques de jeu (validation serveur, message de retour, config robots, actions)
1 jour (24 sept)	Gestion de fin de partie (rejouer, quitter, résultat)
1 jour (25 sept)	Tests unitaires et d'intégration (selon la planification)

5. Planification du plan de tests :

Nom de la classe	Tests (à vérifier)	Type de test
GameController	Création de partie, changement de tour, points attribués, actions, actions invalides, statuts	Unitaire
Game	Début et fin de partie, gagnant, création du robot	Unitaire
Robot	Stats après : attaque, attaque puissante, défense, recharge	Unitaire
MessageHelper	Créer un message, parser le message	Unitaire

6. Remarque

Tous nos diagrammes sont disponibles en annexe de ce document